

运筹学与最优化方法实验报告

陳日康 信息与计算科学 22336049

一、实验要求

- 考虑一个 10 节点的分布式系统。节点 i 有线性测量 $b_i = A_i x + e_i$ ，其中 b_i 为 10 维的测量值， A_i 为 10×200 维的测量矩阵， x 为 200 维的未知稀疏向量且稀疏度为 5， e_i 为 10 维的测量噪声。从所有 b_i 与 A_i 中恢复 x 的一范数规范化最小二乘模型如下：

$$\min \frac{1}{2} \|A_1 x - b_1\|^2 + \dots + \frac{1}{2} \|A_{10} x - b_{10}\|^2 + p \|x\|_1, \text{ 其中 } p \text{ 为非负的正则化参数。}$$

使用邻近点梯度算法、交替方向乘子法与次梯度法求解该问题。在实验中，设 x 的真值中的非零元素服从均值为 0 方差为 1 的高斯分布， A_i 中的元素服从均值为 0 方差为 1 的高斯分布， e_i 中的元素服从均值为 0 方差为 0.1 的高斯分布。对于每种算法，需给出每步计算结果与真值的距离以及每步计算结果与最优解的距离。另外，需要讨论正则化参数 p 对计算结果的影响。

- 使用梯度下降法与随机梯度法，求解 MNIST 数据集上的分类问题，对于每种算法，需要给出每步计算结果在测试集上所对应的分类精度。对于随机梯度法，讨论 mini-batch 大小的影响。可采用 Logistic Regression 模型或神经网络模型。

二、算法原理

- 邻近点梯度法 (Proximal Gradient Method)

邻近点梯度法是一种用于求解带有非光滑正则项的凸优化问题的迭代算法。该方法将目标函数分解为两部分：一个光滑可导函数 $f(x)$ ，和一个非光滑但具有可计算邻近算子的函数 $g(x)$ 。在每一步中，通过以下公式进行更新： $x^{k+1} = \text{prox}_{\eta g}(x^k - \eta \nabla f(x^k))$ ，其中 $\text{prox}_{\eta g}(\cdot)$ 是邻近算子， η 为步长。在本实验中， $f(x)$ 是平方误差项， $g(x)$ 是 ℓ_1 -范数稀疏正则项。该方法具有较强的数值稳定性，尤其适合求解带稀疏约束的问题。

- 交替方向乘子法 (ADMM)

ADMM 是一种用于求解具有可分结构的凸优化问题的迭代方法。其基本形式为 $\min_{x,z} f(x) + g(z)$ ，约束为 $x = z$ 。ADMM 将原问题转化为两个子问题，引入增广拉格朗日函数：

$\mathcal{L}_\rho(x, z, u) = f(x) + g(z) + \frac{\rho}{2} \|x - z + u\|^2 - \frac{\rho}{2} \|u\|^2$ ，其中 $\rho > 0$ 是增广项的系数，用于调节原始变量与引入变量之间的一致性约束在优化过程中的影响强度。算法通过交替优化变量 x 与 z ，再更新对偶变量 u 。通过这种迭代方法可以降低计算复杂度，具备良好的收敛性与稳定性。

3. 次梯度法 (Subgradient Method)

次梯度法是传统梯度下降法在非光滑函数上的推广。对于非光滑但凸的函数 $F(x)$ ，虽然其在某些点不可导，但仍可以通过次梯度 $g^k \in \partial F(x^k)$ 进行更新： $x^{k+1} = x^k - \eta_k g^k$ ，其中 η_k 是步长（可固定或逐步递减）， $\partial F(x)$ 表示 F 在 x 处的次梯度集合。次梯度法简单易实现，但收敛速度较慢，通常为 $O(\frac{1}{\sqrt{k}})$ 。在本实验中， ℓ_1 正则项的次梯度为 $\partial \|x\|_1 = \text{sign}(x)$ ，可以直接代入更新公式中。

4. 梯度下降法 (Gradient descent, GD)

梯度下降法是一种基于一阶导数的信息的最优化方法，广泛应用于分类模型的训练中。其核心目标是最小化损失函数，从而找到最优的参数 w 。梯度下降的基本思想是：在每一次迭代中，计算损失函数对参数的梯度，然后沿梯度的反方向进行更新参数，更新公式为： $w^{(k+1)} = w^{(k)} - \eta \cdot \nabla L(w^{(k)})$ ，其中， η 是学习率，控制步长； $\nabla L(w^{(k)})$ 是损失函数在第 k 次迭代时的梯度。梯度下降法每次迭代都使用整个训练集来计算一次梯度，因此收敛过程较稳定，但计算成本较高。

5. 随机梯度法 (Stochastic Gradient Descent, SGD)

随机梯度法是在梯度下降法的基础上发展而来的，旨在提高大规模数据集下的优化效率。与标准梯度下降法不同，SGD 在每次迭代中并不使用整个训练集计算梯度，而是从中随机选取一个样本或一个小批次 (mini-batch) 来估计当前的梯度，并据此更新参数。其更新公式如下： $w^{(k+1)} = w^{(k)} - \eta \cdot \nabla \ell(w^{(k)}; x_i, y_i)$ ，其中 (x_i, y_i) 表示当前选中的样本， $\ell(\cdot)$ 是其对应的损失函数。由于每轮更新仅依赖部分样本，SGD 的单步计算代价远低于标准梯度下降，适用于大规模或在线学习场景。但也正因为如此，其更新方向仅为目标函数梯度的近似，训练过程可能伴随较强震荡，收敛速度更依赖于学习率和样本顺序。

三、算法实现

1. 邻近点梯度法 (Proximal Gradient Method，实现于 `proximal_gradient.py`)

(i). 函数 `generate_data` 生成稀疏信号 `x_true`、对应的观测矩阵 `A_list` 以及加入噪声的观测向量 `b_list`。

```
def generate_data(n_nodes=10, m=10, n=200, sparsity=5, noise_std=0.1):
    np.random.seed(42)      # 设置随机种子
    x_true = np.zeros(n)     # 初始化稀疏信号
    idx = np.random.choice(n, sparsity, replace=False)  # 随机选择非零元素的位置
    x_true[idx] = np.random.randn(sparsity)             # 给稀疏向量赋随机值
    A_list = [np.random.randn(m, n) for _ in range(n_nodes)]
    e_list = [noise_std * np.random.randn(m) for _ in range(n_nodes)]
    b_list = [A @ x_true + e for A, e in zip(A_list, e_list)]
    return A_list, b_list, x_true
```

(ii). 函数 `solve_with_cvxpy` 利用 CVXPY 求解 L1 正则化的最小二乘问题，作为最优解 `x_opt` 的参考值。

```
def solve_with_cvxpy(A_list, b_list, p):
    n = A_list[0].shape[1]      # 变量维度
    x = cp.Variable(n)          # 定义优化变量 x

    # 构造目标函数，包括误差项和 L1 正则化
    obj = 0.5 * sum([cp.sum_squares(A @ x - b) for A, b in zip(A_list,
b_list)]) + p * cp.norm1(x)

    # 构造并求解优化问题
    cp.Problem(cp.Minimize(obj)).solve()
    return x.value
```

(iii). 函数 `prox_L1(v, tau)` 实现 L1 正则项对应的 proximal operator，即 soft-thresholding 操作，用于稀疏约束。

```
def prox_L1(v, tau):
    return np.sign(v) * np.maximum(np.abs(v) - tau, 0)
```

(iv). 函数 `proximal_gradient` 使用 Proximal Gradient Method 解目标函数，每步记录与真值 `x_true` 和最优解 `x_opt` 的距离。

```
def proximal_gradient(A_list, b_list, p, x_true, x_opt, stepsize=1e-3,
max_iter=1000):
    n = A_list[0].shape[1]      # 变量维度
    x = np.zeros(n)             # 初始解为零
    d_true = []
    d_opt = []

    def grad(x):                 # 计算目标函数的梯度
        return sum([A.T @ (A @ x - b) for A, b in zip(A_list, b_list)])

    for _ in range(max_iter):
        x = prox_L1(x - stepsize * grad(x), stepsize * p)    # 计算梯度并进行更新

        # 记录误差
        d_true.append(np.linalg.norm(x - x_true))
        d_opt.append(np.linalg.norm(x - x_opt))

    return d_true, d_opt
```

2. 交替方向乘子法 (ADMM, 实现于 `admm.py`)

函数 `admm_solver` 实现交替方向乘子法 (ADMM) 以分布式方式求解带 L1 正则化的最小二乘问题, 记录每一步迭代结果与真值及最优解的距离。

```
def admm_solver(A_list, b_list, p, x_true, x_opt, rho=1.0, max_iter=1000):
    n_nodes = len(A_list)
    n = A_list[0].shape[1]

    # 初始化
    x_list = [np.zeros(n) for _ in range(n_nodes)] # 局部变量
    z = np.zeros(n) # 全局变量
    u_list = [np.zeros(n) for _ in range(n_nodes)] # 对偶变量

    d_true = [] # 距离 x_true 的误差
    d_opt = [] # 距离最优解 x_opt 的误差

    # 提前计算每个节点需要用到的逆矩阵
    inv_matrices = [np.linalg.inv(A.T @ A + rho * np.eye(n)) for A in A_list]

    for _ in range(max_iter):
        for i in range(n_nodes): # 更新局部变量 x_i
            A, b = A_list[i], b_list[i]
            u = u_list[i]
            rhs = A.T @ b + rho * (z - u)
            x_list[i] = inv_matrices[i] @ rhs

        # 更新共享变量 z
        x_u_avg = np.mean([x_list[i] + u_list[i] for i in range(n_nodes)],
axis=0)
        z = np.sign(x_u_avg) * np.maximum(np.abs(x_u_avg) - p / (n_nodes *
rho), 0)

        for i in range(n_nodes): #更新对偶变量 u_i
            u_list[i] += x_list[i] - z

        # 记录每次迭代的误差
        x_avg = np.mean(x_list, axis=0)
        d_true.append(np.linalg.norm(x_avg - x_true))
        d_opt.append(np.linalg.norm(x_avg - x_opt))

    return d_true, d_opt
```

3. 次梯度法 (Subgradient Method, 实现于 `subgradient.py`)

函数 `subgradient_solver` 实现次梯度法, 通过计算损失函数和 L1 正则项的次梯度对参数进行更新, 追踪与真解及最优解的距离。

```

def subgradient_solver(A_list, b_list, p, x_true, x_opt, stepsize=1e-4,
max_iter=1000):
    n = A_list[0].shape[1]
    x = np.zeros(n)    # 初始化解
    d_true = []
    d_opt = []

    for k in range(max_iter):
        # Compute subgradient
        grad = np.zeros(n)
        for A, b in zip(A_list, b_list):
            grad += A.T @ (A @ x - b)    # 梯度部分
        grad += p * np.sign(x)           # L1 正则项的次梯度

        x -= stepsize * grad              # 参数更新

        d_true.append(np.linalg.norm(x - x_true))    # 距离真值
        d_opt.append(np.linalg.norm(x - x_opt))      # 距离最优解

    return d_true, d_opt

```

4. 梯度下降法 (Gradient descent, 实现于 mnist_logistic.py)

(i). 函数 `evaluate` 计算模型在测试集上的分类准确率，用于评估每次训练的性能。

```

@torch.no_grad()    # 在评估时不计算梯度
def evaluate(model, test_loader):
    model.eval()     # 设置模型为评估模式
    correct = total = 0
    for x, y in test_loader:
        out = model(x)    # 模型预测输出
        preds = out.argmax(dim=1)    # 获取预测的类别 (最大值索引)
        correct += (preds == y).sum().item()    # 计算预测正确的数量
        total += y.size(0)    # 总样本数
    return correct / total

```

(ii). 类 `LogisticRegression` 用于将 MNIST 图像展平后输入线性分类器，输出 10 类数字的预测分布。其中 `self.linear` 是逻辑回归的核心，对展平后的图像输入做线性变换。`forward()` 函数定义了前向传播逻辑：图像展平后送入线性层输出 logits。

```

class LogisticRegression(nn.Module):
    def __init__(self):
        super().__init__()
        self.linear = nn.Linear(28 * 28, 10)    # 全连接层

    def forward(self, x):
        x = x.view(-1, 28 * 28)    # 将输入图层展平成一维向量
        return self.linear(x)

```

(iii). 函数 `train_gd` 使用标准梯度下降法（GD）训练模型，并在每轮训练后评估准确率。

```

def train_gd(epochs=10, lr=0.1):
    train_loader, test_loader = load_mnist(batch_size=60000)    # 加载数据
    model = LogisticRegression()    # 初始化模型
    criterion = nn.CrossEntropyLoss()    # 损失函数
    acc_list = []    # 记录每轮的准确率

    for epoch in range(epochs):
        model.train()    # 设置模型为训练模式
        for x, y in train_loader:
            out = model(x)    # 前向传播
            loss = criterion(out, y)    # 计算损失

            model.zero_grad()    # 清空梯度
            loss.backward()    # 反向传播，计算梯度
            with torch.no_grad():    # 关闭梯度计算
                for param in model.parameters():
                    param -= lr * param.grad    # 更新权重

        acc = evaluate(model, test_loader)    # 计算测试集上的准确率
        acc_list.append(acc)    # 记录准确率
        print(f"[GD] Epoch {epoch+1}: Test Accuracy = {acc:.4f}")
    return acc_list

```

5. 随机梯度法（Stochastic Gradient Descent，实现于 `mnist_logistic.py`）

函数 `train_sgd` 使用随机梯度下降法（SGD）训练模型，支持不同的 batch size，并评估每轮的准确率。

```

def train_sgd(batch_size=64, epochs=10, lr=0.1):
    train_loader, test_loader = load_mnist(batch_size=batch_size)
    model = LogisticRegression()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=lr)    # 使用 SGD 优化器

```

```

acc_list = []

for epoch in range(epochs):
    model.train()
    for x, y in train_loader:
        optimizer.zero_grad()      # 清空梯度
        out = model(x)             # 向前传播
        loss = criterion(out, y)    # 计算损失
        loss.backward()            # 反向传播
        optimizer.step()

    acc = evaluate(model, test_loader) # 计算测试集上的准确率
    acc_list.append(acc)
    print(f"[SGD bs={batch_size}] Epoch {epoch+1}: Test Accuracy = {acc:.4f}")
    return acc_list

```

梯度下降法与随机梯度法两个算法共享相同的数据加载与模型结构，分别通过 `train_gd` 与 `train_sgd` 函数实现不同的优化策略，便于在同一环境下进行公平对比与实验验证。

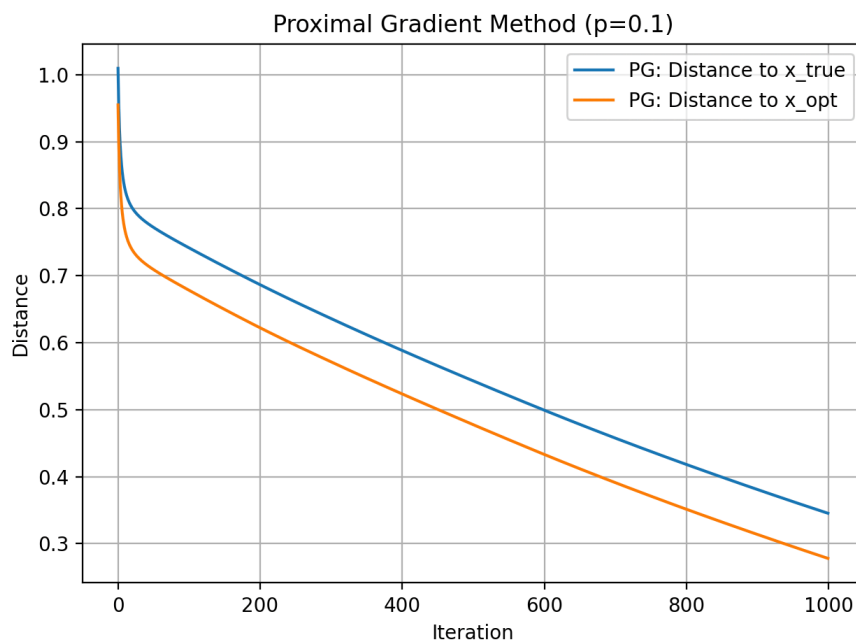
四、实验结果与分析

实验1：

(1). $p=0.1$:

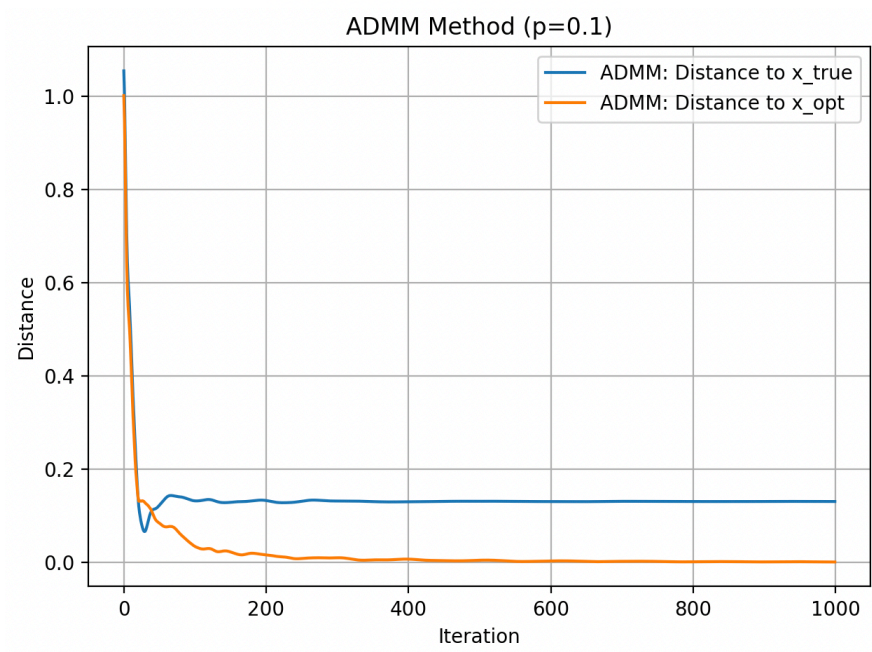
(i). 邻近点梯度法：

Proximal Gradient Method $p=0.1$:
 最终迭代距离真值 = 0.3454
 最终迭代距离最优解 = 0.2781



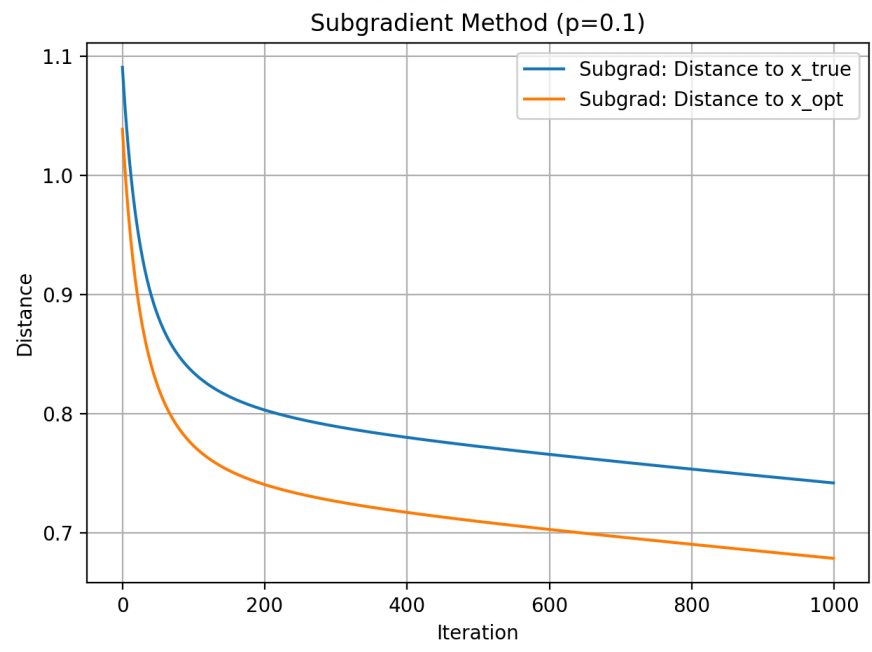
(2). 交替方向乘子法：

ADMM Method $p=0.1$:
最终迭代距离真值 = 0.1303
最终迭代距离最优解 = 0.0005



(iii). 次梯度法：

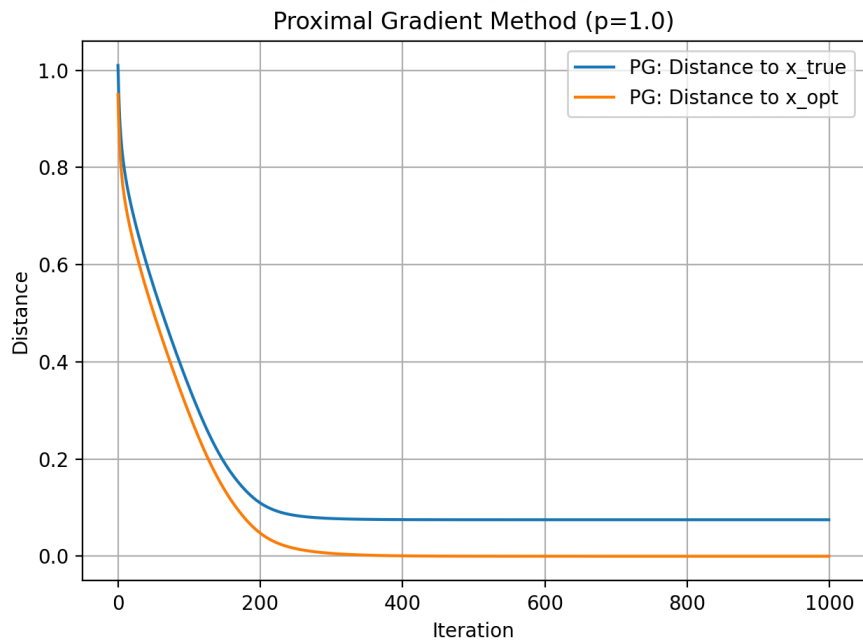
Subgradient Method $p=0.1$:
最终迭代距离真值 = 0.7421
最终迭代距离最优解 = 0.6789



(2). $p=1.0$:

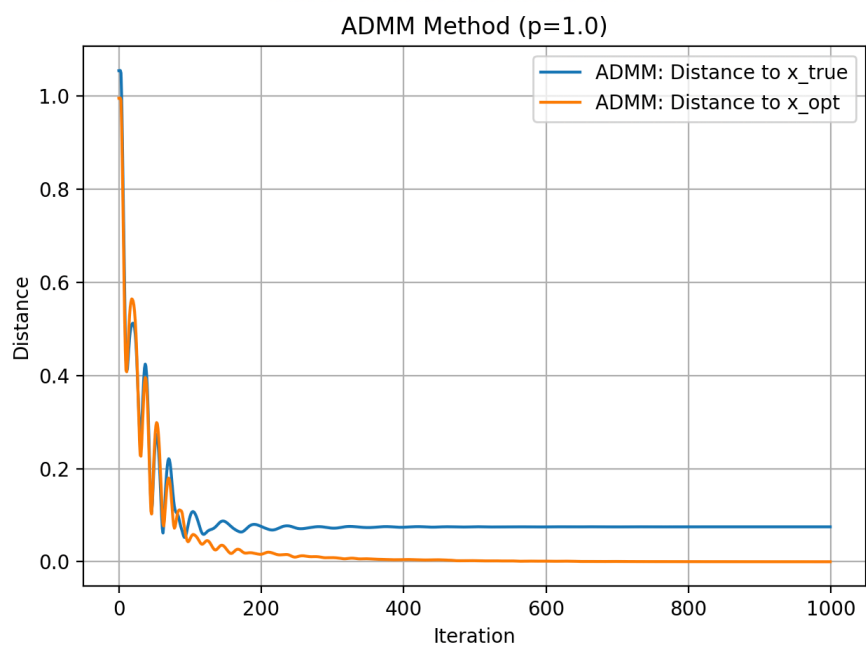
(i). 邻近点梯度法：

Proximal Gradient Method $p=1.0$:
最终迭代距离真值 = 0.0750
最终迭代距离最优解 = 0.0000



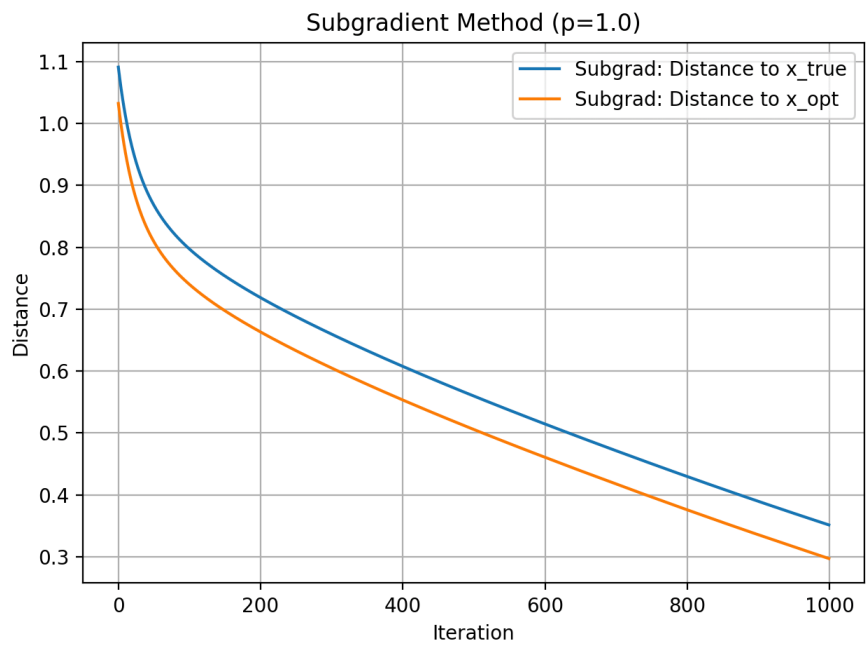
(ii). 交替方向乘子法：

ADMM Method $p=1.0$:
最终迭代距离真值 = 0.0750
最终迭代距离最优解 = 0.0001



(iii). 次梯度法：

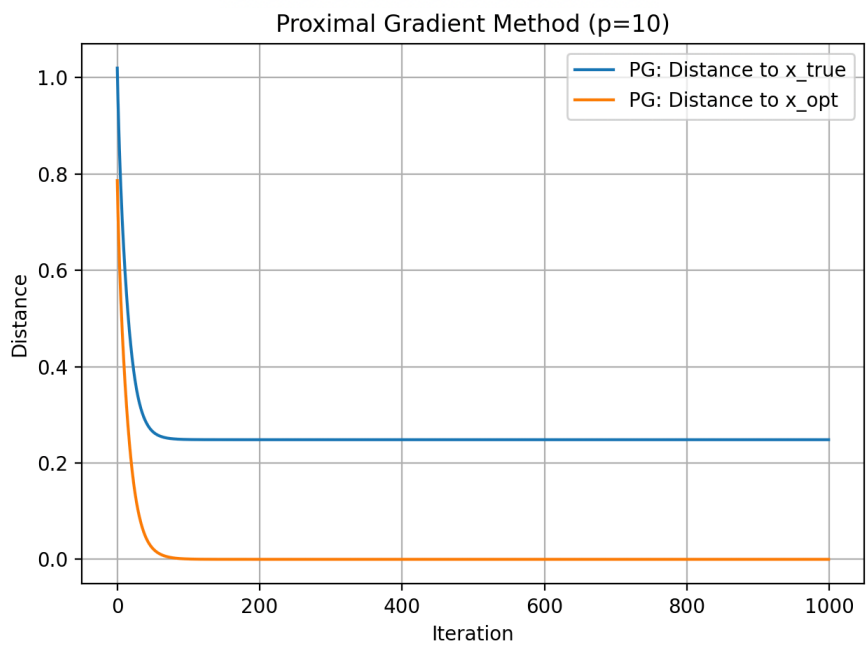
Subgradient Method $p=1.0$:
最终迭代距离真值 = 0.3516
最终迭代距离最优解 = 0.2974



(3). $p=10$:

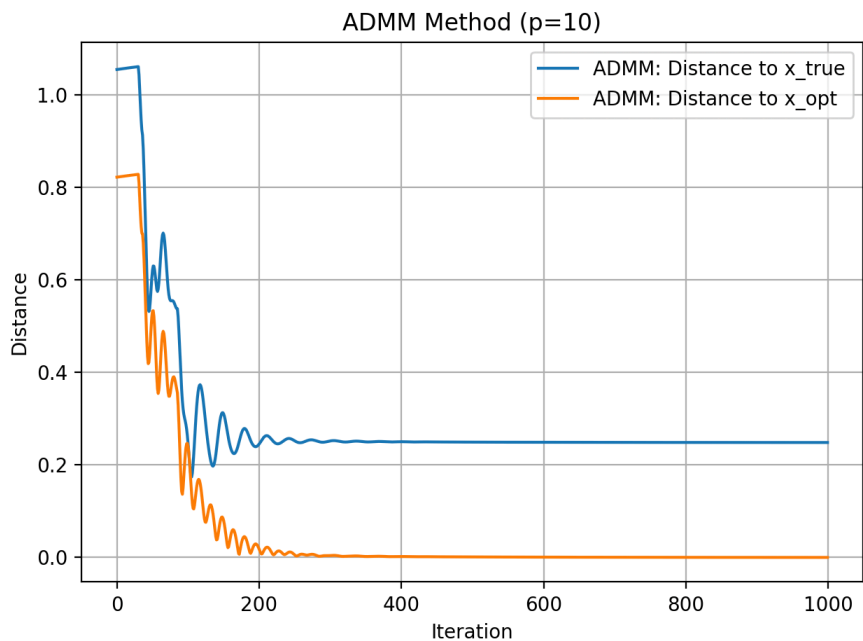
(i). 邻近点梯度法：

Proximal Gradient Method $p=10$:
 最终迭代距离真值 = 0.2484
 最终迭代距离最优解 = 0.0000



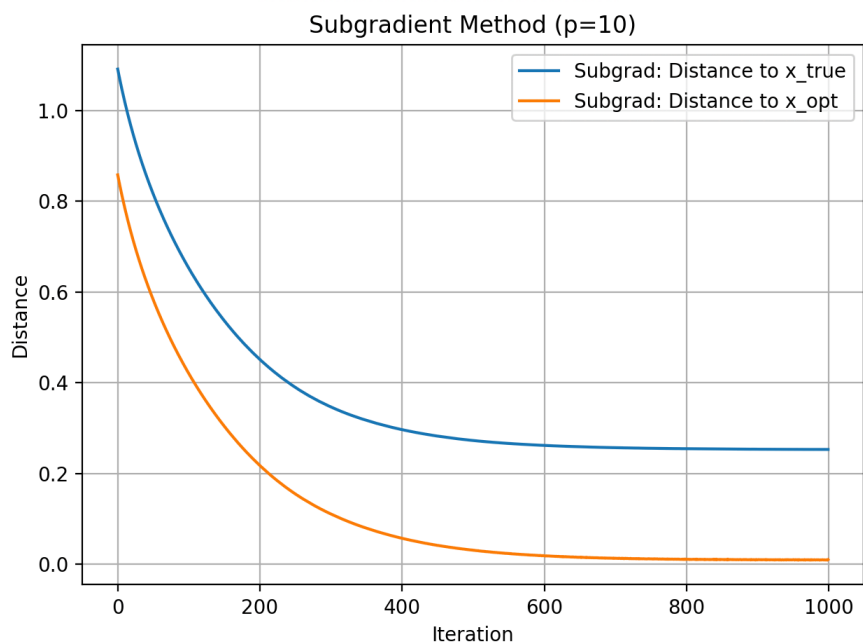
(ii). 交替方向乘子法：

ADMM Method $p=10$:
 最终迭代距离真值 = 0.2485
 最终迭代距离最优解 = 0.0001



(iii). 次梯度法：

Subgradient Method $p=10$:
 最终迭代距离真值 = 0.2532
 最终迭代距离最优解 = 0.0106



(4). 正则化参数 p 对计算结果的影响：

从实验结果来看，随着 p 的增大，优化模型更倾向于选择稀疏解，即保留更少的非零元素。这种趋势在 ADMM 和邻近点梯度法中表现得尤为明显：当 p 较小时（如 0.01 或 0.1），解对观测数据的拟合更充分，但保留了较多冗余成分；而当 p 增大到 1.0 时，解的稀疏性明显增强，同时与真实解之间的距离也随之变大，体现出正则化对信息压缩的影响。

从收敛过程来看，ADMM 的表现最为稳定，不同 p 值下均能在较短迭代内逼近最优解。邻近点梯度法对 p 的选择较为敏感， p 过大时会导致收敛速度下降。次梯度法在步长固定的前提下，随 p 增大会出现更明显的震荡和数值不稳定，其误差曲线波动明显高于前两种方法。

适中的正则化参数（如 $p=0.1$ ）在保证一定稀疏性的同时，也维持了较好的解质量。过小的 p 会削弱稀疏结构的作用，过大的 p 则可能过度惩罚非零项，导致有用信息的丢失。

实验2：

(1). 实验结果：

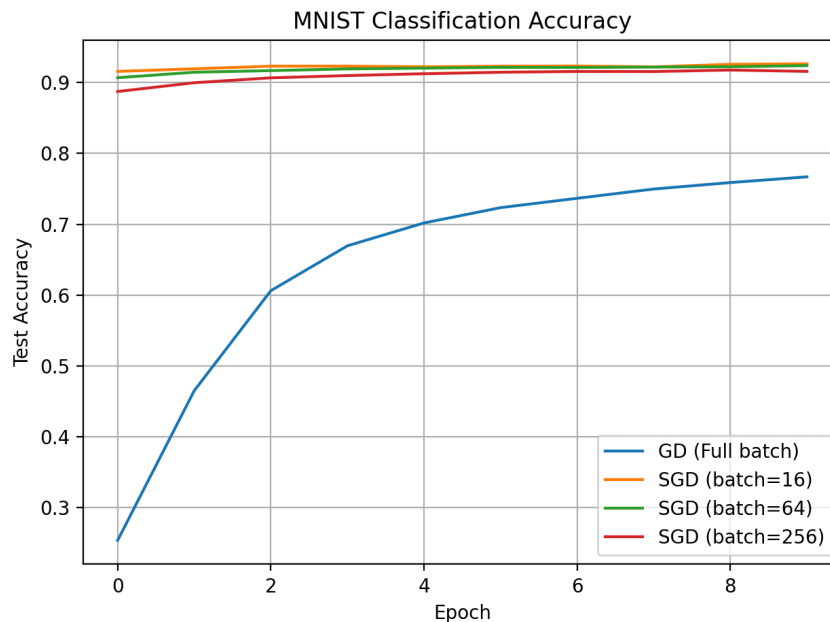
```
[GD] Epoch 1: Test Accuracy = 0.2535
[GD] Epoch 2: Test Accuracy = 0.4649
[GD] Epoch 3: Test Accuracy = 0.6061
[GD] Epoch 4: Test Accuracy = 0.6695
[GD] Epoch 5: Test Accuracy = 0.7018
[GD] Epoch 6: Test Accuracy = 0.7234
[GD] Epoch 7: Test Accuracy = 0.7366
[GD] Epoch 8: Test Accuracy = 0.7497
[GD] Epoch 9: Test Accuracy = 0.7588
[GD] Epoch 10: Test Accuracy = 0.7669
```

```
[SGD bs=16] Epoch 1: Test Accuracy = 0.9158
[SGD bs=16] Epoch 2: Test Accuracy = 0.9194
[SGD bs=16] Epoch 3: Test Accuracy = 0.9232
[SGD bs=16] Epoch 4: Test Accuracy = 0.9232
[SGD bs=16] Epoch 5: Test Accuracy = 0.9223
[SGD bs=16] Epoch 6: Test Accuracy = 0.9232
[SGD bs=16] Epoch 7: Test Accuracy = 0.9234
[SGD bs=16] Epoch 8: Test Accuracy = 0.9222
[SGD bs=16] Epoch 9: Test Accuracy = 0.9258
[SGD bs=16] Epoch 10: Test Accuracy = 0.9265
```

```
[SGD bs=64] Epoch 1: Test Accuracy = 0.9069
[SGD bs=64] Epoch 2: Test Accuracy = 0.9146
[SGD bs=64] Epoch 3: Test Accuracy = 0.9169
[SGD bs=64] Epoch 4: Test Accuracy = 0.9193
[SGD bs=64] Epoch 5: Test Accuracy = 0.9204
[SGD bs=64] Epoch 6: Test Accuracy = 0.9215
[SGD bs=64] Epoch 7: Test Accuracy = 0.9214
[SGD bs=64] Epoch 8: Test Accuracy = 0.9219
[SGD bs=64] Epoch 9: Test Accuracy = 0.9224
[SGD bs=64] Epoch 10: Test Accuracy = 0.9239
```

```
[SGD bs=256] Epoch 1: Test Accuracy = 0.8874
[SGD bs=256] Epoch 2: Test Accuracy = 0.8998
[SGD bs=256] Epoch 3: Test Accuracy = 0.9067
[SGD bs=256] Epoch 4: Test Accuracy = 0.9099
[SGD bs=256] Epoch 5: Test Accuracy = 0.9125
[SGD bs=256] Epoch 6: Test Accuracy = 0.9146
[SGD bs=256] Epoch 7: Test Accuracy = 0.9157
[SGD bs=256] Epoch 8: Test Accuracy = 0.9156
[SGD bs=256] Epoch 9: Test Accuracy = 0.9177
[SGD bs=256] Epoch 10: Test Accuracy = 0.9158
```

其中，[GD] 表示梯度下降法，[SGD] 表示随机梯度法，bs 表示 batch 的大小（16、64、256）。



从实验图像清楚地展示了梯度下降法（GD）与不同 mini-batch 大小下的随机梯度下降法（SGD）在 MNIST 分类任务中的性能对比。其中，GD 采用全量样本更新，每次迭代计算开销较大，收敛速度明显慢于 SGD，但精度提升过程较为平稳。而 SGD 方法在前几轮就快速达到较高精度，整体表现优于 GD，说明在实际应用中更具效率优势。特别地，SGD (batch=64) 的表现最为理想，在精度和收敛稳定性之间取得了良好平衡，适合作为默认的 batch 设置。

(2). 对于随机梯度法，mini-batch 大小的影响：

从计算效率的角度来看，较小的 batch 能引入更多噪声，使优化路径更随机，有助于逃离局部极小值并提升泛化能力，但计算效率较低；较大的 batch 能提供更稳定的梯度估计，允许使用更大的学习率加速收敛，但可能陷入尖锐极小值，泛化性变差，同时占用更多内存。

五、实验总结与心得

本次实验实现稀疏最小二乘问题下的三种优化方法与对比：邻近点梯度法、交替方向乘子法（ADMM）和次梯度法。它们在迭代速度和精度上表现各异。ADMM 通过分解变量并交替更新，收敛较快且稳定性好；邻近点梯度法利用软阈值对 ℓ_1 项做近端操作，整体表现中规中矩；而次梯度法方法由于方向不唯一，收敛慢且震荡大。实验二则考察了随机梯度下降在不同 mini-batch 设置下的分类精度，发现小 batch 在初期更新频繁、上升较快，但波动也更明显；大 batch 训练更平稳，但收敛速度慢，最终精度差别不大。

通过本次实验，对稀疏优化中几种常见算法的原理与实现方式有了更系统的理解。实验过程中，观察了不同算法在收敛速度、精度及稳定性方面的表现，对 ADMM 的迭代机制和 Proximal 方法中的软阈值操作有了更直观的认识。此外，在分类实验中，体会到了 mini-batch 大小对训练动态与精度的实际影响。整体而言，本次实验加深了我对凸优化理论与实际算法行为之间联系的理解，也提高了我对优化问题数值解法的掌握程度。